

FINAL PROJECT REPORT

AI Smart Tutor using Advanced Prompt Engineering Techniques

Rancang Bangun Website AI Smart Tutor sebagai Media Tanya Jawab Pembelajaran Interaktif

AI Smart Tutor Website as an Interactive Learning Question-Answering Media

Prepared by	Ivansyah Putra
Institution / Campus	STMIK Kaputama Binjai
Class / Course	Google Prompt Engineering
Project Type	Web-based AI Educational Chatbot
Main Technology	Python Flask, Gemini API, HTML, CSS, JavaScript
Data Approach	No database; browser Local Storage for chat history
Report Version	Final Project Report aligned with WBS M1-M6

This report documents the complete design, implementation, testing, and final outcome of the AI Smart Tutor project.

1. Executive Summary

AI Smart Tutor is a web-based educational chatbot developed as part of the Google Prompt Engineering class. The system is designed to provide interactive learning support through question answering, concept explanation, study plan generation, quiz generation, and answer evaluation. The project uses Python Flask as the backend, Gemini API as the AI model service, and HTML, CSS, and JavaScript as the frontend technologies.

The final website supports four AI Tutor Modes: Question Answering, Study Plan Generator, Quiz Generator, and Answer Evaluator. It also supports three learning levels: Beginner, Intermediate, and Advanced. Each mode uses a different prompt template so the AI output becomes more structured, consistent, relevant, and aligned with the user task. The website does not use a database; chat history is stored locally in the browser using Local Storage.

2. Background

Students often need learning assistance that is fast, structured, and easy to understand. However, normal search engines or general AI chatbots may provide broad answers that are not always organized for learning. This project addresses that issue by building an AI-powered tutor website that guides users with structured educational responses.

The project also applies advanced prompt engineering techniques. Instead of using one general prompt for every task, the system uses mode-based prompts. Each mode has its own response structure, task instructions, and learning-level adaptation. This makes the chatbot more reliable for educational use.

3. Problem Statement

The main problem addressed in this project is the need for a structured AI learning assistant that can provide educational answers in a consistent format. Students may ask different types of learning requests, such as asking a concept question, requesting a study plan, generating quiz questions, or evaluating an answer. A single unstructured chatbot response is often not enough to support these different learning tasks.

- Students need clear academic explanations that are easy to understand.
- Students need learning support that adapts to different ability levels.
- Students need tools to practice through quizzes and structured study plans.
- Students need feedback and scoring for their written answers.
- The system needs to work without a database while still keeping chat history locally.

4. Project Objectives

- To develop a web-based AI Smart Tutor chatbot using Python Flask and Gemini API.
- To implement structured answer generation for academic questions.
- To provide multi-level explanation support through Beginner, Intermediate, and Advanced levels.
- To develop Study Plan Generator and Quiz Generator modes.
- To develop an Answer Evaluator mode with score, strengths, weaknesses, corrected answer, and learning tips.
- To apply prompt comparison and prompt optimization through mode-based prompt engineering.
- To create a modern website interface suitable for project demonstration and video recording.

5. Project Scope

The scope of this project focuses on creating an AI-based educational website that supports academic learning interactions. The system is not designed as a full Learning Management System and does not include user accounts or a database. Instead, it focuses on prompt engineering, AI response quality, structured outputs, and browser-based chat history.

Included Scope	Description
Web chatbot interface	A modern website where users can type academic questions and receive AI responses.
Gemini API integration	The backend sends structured prompts to Gemini API and returns responses to the frontend.
AI Tutor Modes	Question Answering, Study Plan Generator, Quiz Generator, and Answer Evaluator.
Learning levels	Beginner, Intermediate, and Advanced response adaptation.
Chat history	Browser Local Storage is used to keep chat history after refresh.
Project explanation	The website includes an explanation section for demo and presentation purposes.

6. System Features

Feature	Description	WBS Alignment
Question Answering	Answers academic questions using structured responses and context-aware explanation.	M2
Concept Explanation	Simplifies complex topics into clear and understandable explanations.	M2
Learning Level Selection	Adjusts explanations based on Beginner, Intermediate, or Advanced level.	M2
Study Plan Generator	Generates day-by-day learning plans for selected topics.	M3
Quiz Generator	Creates quiz questions, answer keys, and learning tips.	M3
Answer Evaluator	Evaluates user answers with score, strengths, weaknesses, corrected answer, and feedback.	M4
Prompt Optimization	Uses mode-based prompt templates to improve response quality.	M5
Project Explanation	Provides project details, technologies, workflow, and prompt optimization explanation inside the website.	M6

7. Work Breakdown Structure Alignment

Milestone	Requirement	Implementation Result
M1	Project setup and initial prompt exploration	Flask project, Gemini API setup, and initial AI tutor prompts were implemented.
M2	Structured answer generator and multi-level explanation generator	Structured responses and Beginner, Intermediate, Advanced levels were implemented.
M3	Study Plan Generator and Quiz Generator	Both modes were added in the AI Tutor Mode selector.
M4	Answer Evaluator (Examiner AI)	Answer Evaluator mode provides score, feedback, corrected answer, and learning tip.
M5	Prompt Comparison Engine and Optimization	Mode-based prompt templates and prompt optimization explanation were implemented.
M6	Finalization, documentation, and presentation	Final website, proposal, prompt library, report, slides, and demo materials are prepared.

8. Technology Stack

Technology	Purpose
Python Flask	Backend web framework for routing, rendering pages, and handling API requests.
Gemini API	AI model service for generating educational responses.
HTML	Defines the structure of the website.
CSS	Creates the modern visual design, responsive layout, cards, chatbot UI, and project explanation section.
JavaScript	Handles chatbot interaction, fetch requests, mode selection, local storage, and UI events.
Local Storage	Stores chat history locally in the user browser without a database.

9. System Design and Workflow

The AI Smart Tutor follows a simple client-server architecture. The user interacts with the frontend website. When the user sends a message, JavaScript sends the input, selected mode, and selected learning level to the Flask backend through a POST request. Flask builds a mode-specific prompt and sends it to Gemini API. Gemini generates a response, and the response is returned to the website and displayed inside the chatbot interface.

1. User selects an AI Tutor Mode.
2. User selects a learning level.
3. User types an academic input and clicks Send.

4. JavaScript sends the request to the Flask /ask route.
5. Flask builds a structured prompt based on mode and level.
6. Flask sends the prompt to Gemini API.
7. Gemini returns an optimized educational response.
8. The response is displayed in the chatbot and saved to Local Storage.

Input Component	Processed By	Output
User question or task	JavaScript and Flask	Request payload sent to backend
AI Tutor Mode	Prompt builder in Flask	Selected prompt template
Learning Level	Prompt builder in Flask	Adjusted response difficulty
Gemini API response	Flask and JavaScript	Displayed chatbot answer
Chat history	Browser Local Storage	Stored conversation after refresh

10. AI Tutor Modes

Mode	Purpose	Expected Output
Question Answering	To answer academic questions clearly.	Answer, context-aware explanation, concept explanation, and learning guidance.
Study Plan Generator	To generate a structured learning schedule.	Study topic, learning goal, day-by-day plan, guidance, and learning tip.
Quiz Generator	To generate practice questions.	Quiz topic, instructions, practice questions, answer key, and learning tip.
Answer Evaluator	To evaluate user answers like an examiner.	Score, strengths, weaknesses, corrected answer, guidance, and learning tip.

11. Prompt Engineering Approach

The project applies prompt engineering techniques to control the AI output. The prompt includes role definition, task instruction, learning-level adaptation, and structured output requirements. This ensures that Gemini produces responses that are consistent and suitable for education.

Technique	Implementation
Role-based prompting	Gemini is instructed to act as an AI Smart Tutor or Examiner AI.
Structured output prompting	Each mode has a fixed response structure.
Mode-based prompting	Each AI Tutor Mode uses a different prompt template.
Learning-level adaptation	Responses are adjusted for Beginner, Intermediate, or Advanced users.
Rubric-based prompting	Answer Evaluator uses score, strengths, weaknesses, and corrected answer.
Few-shot prompting concept	Example patterns are documented in the Prompt Template

Technique	Implementation
	Library.

12. Prompt Comparison and Optimization

The prompt comparison process compares different prompt structures based on response clarity, relevance, consistency, and educational value. The final project uses optimized prompts that are more task-specific and structured compared with a single general prompt.

Prompt Type	Strength	Weakness	Final Decision
General prompt	Simple and easy to use.	Responses may be inconsistent and less structured.	Not used as final main prompt.
Structured prompt	Produces organized educational responses.	Requires longer prompt design.	Used for Question Answering.
Mode-based prompt	Matches response format to user task.	Requires multiple templates.	Used as final approach.
Rubric-based evaluator prompt	Provides measurable score and feedback.	Needs clear user answer input.	Used for Answer Evaluator.

13. Implementation Summary

The website was implemented with a clean and modern interface. The hero section introduces the AI Smart Tutor. The features section explains the core functionalities. The chatbot section allows the user to choose an AI Tutor Mode and Learning Level. The Project Information section explains the project overview, technologies, workflow, prompt optimization, and final result.

- The backend was built using Flask.
- The Gemini API key was stored securely in the .env file.
- The frontend was built using HTML, CSS, and JavaScript.
- The chatbot communicates with the backend through a POST request to the /ask route.
- Chat history is saved using Local Storage, not a database.

14. Testing and Results

Testing was conducted by trying each AI Tutor Mode and checking whether the output matched the expected format. The website was also tested by refreshing the browser to confirm that chat history was retained by Local Storage.

Test Case	Input Example	Expected Result	Status
Question Answering	What is database?	Structured academic explanation with learning support.	Passed
Study Plan Generator	Create a 5-day study plan for HTML and CSS.	Day-by-day study plan and learning tip.	Passed
Quiz Generator	Create 5 quiz questions	Five questions and answer	Passed

Test Case	Input Example	Expected Result	Status
	about information systems.	key.	
Answer Evaluator	Question plus user answer about database.	Score, feedback, strengths, weaknesses, corrected answer.	Passed
Learning Level	Same topic with Beginner and Advanced levels.	Different explanation depth based on level.	Passed
Chat History	Refresh after conversation.	Previous messages remain visible.	Passed

15. Final Output

The final output of this project is a working AI Smart Tutor website that supports interactive learning through multiple AI Tutor Modes. The system is suitable for educational question answering, concept explanation, study planning, quiz creation, and answer evaluation. The project is also ready for video demonstration and submission with supporting documents.

- A functional Flask web application.
- A modern chatbot interface in English.
- Gemini API integration for AI responses.
- Four AI Tutor Modes aligned with WBS requirements.
- Three learning levels for personalized guidance.
- Project explanation section inside the website.
- Chat history support using Local Storage.

16. Limitations

- The project does not use a database, so chat history is only stored in the browser where the website is used.
- The quality of AI responses depends on the Gemini API response and prompt clarity.
- The website does not include user login or online deployment in this version.
- Quiz answers are generated by the AI and should be reviewed if used for formal assessment.

17. Future Development

- Add user login and database storage for long-term chat history.
- Add downloadable quiz results and study plans.
- Add a dedicated prompt comparison dashboard.
- Add deployment to a cloud platform so the website can be accessed online.
- Add analytics to measure learning progress over time.

18. Conclusion

The AI Smart Tutor project successfully implements an AI-powered educational chatbot using Python Flask and Gemini API. The system provides structured academic question answering, multi-level explanations, study plan generation, quiz generation, and answer evaluation. The project also demonstrates advanced

prompt engineering through role-based prompting, structured output prompting, mode-based prompting, rubric-based prompting, and prompt optimization.

Overall, the project meets the WBS requirements from M1 to M6 and produces a functional, modern, and presentable learning assistant website. The final system is suitable for demonstration, documentation, and submission as a Google Prompt Engineering final project.